

PS 26052 — ANC Prototype Development

Software Implementation Task — Modules 6 to 7

Measured/Recorded Acoustic Integration → Standardized Evaluation → AI-Ready Dataset Interface

Build two continuous, modular implementation modules that complete the non-AI ANC foundation. The output of each module must be reusable by the next phase, where Module 8 begins AI-based residual-noise suppression.

Module 1 Signal Foundation: Architecture → Generation → FIR/LTI Processing [COMPLETE]

Module 2 Statistical Estimation: Correlation → Wiener/MMSE Benchmark [COMPLETE]

Module 3 Adaptive Filtering Laboratory [COMPLETE]

Module 4 Digital Feedforward ANC Plant [COMPLETE]

Module 5 Secondary-Path Identification [COMPLETE]

Module 6 Recorded/Measured Acoustic ANC Integration

↓

Module 7 Standardized Evaluation & AI-Ready ANC Dataset

↓

Module 8 AI Residual-Noise Suppression / Learned Enhancement

This continuation preserves the same implementation philosophy used in Modules 1–5: configuration-driven experiments, reusable interfaces, saved artifacts, visualization, testing, explicit acceptance conditions, and strict separation of simulation, recorded-data, and future live-hardware paths.

1. Overview

Modules 3–5 established a working adaptive ANC simulation framework: online LMS/NLMS adaptation, a digital feedforward ANC plant, filtered-x adaptation, and secondary-path identification. The next engineering boundary is not AI. Before adding learned models, the project should be able to operate on reproducible recorded/measured data and evaluate every classical ANC baseline under standardized conditions.

Implementation pipeline

```
MODULE 5 OUTPUT
excitation + measured response → SecondaryPathModel → ■(z)
↓
MODULE 6
recorded / measured reference + disturbance + calibrated path data
→ offline ANC replay → FxLMS/FxNLMS → residual e[n]
↓
MODULE 7
scenario manifest → standardized metrics → baseline comparison
→ reproducible windows/features/targets + metadata
↓
MODULE 8
AI model consumes the organized non-AI pipeline outputs
```

Module philosophy

```
INPUT
↓
PROCESSING / ALGORITHM
↓
OUTPUT ARTIFACT
↓
ACCEPTANCE TEST
```

No module should become a one-off notebook or experiment-specific script. The same interfaces should support controlled synthetic data now, recorded audio next, and live hardware later. AI must consume stable artifacts rather than reaching directly into ad-hoc experiment code.

Why two modules before AI?

Module 6 closes the gap between an idealized simulated plant and reproducible recorded/measured operation. Module 7 closes the gap between raw experiments and a clean, comparable, versioned data/evaluation interface. After Module 7, the project has a coherent baseline package that an AI model can consume without replacing the validated DSP/ANC pipeline.

2. Shared Notation and Data Contracts

Symbol / object	Meaning
$x[n]$	Reference signal used by the ANC controller.
$d[n]$	Primary disturbance at the error location or controlled target-domain equivalent.
$y[n]$	Controller output / anti-noise command.
$e[n]$	Residual/error signal after secondary-path propagation and cancellation.
$P(z)$	Primary path.
$S(z)$	True/measured physical secondary path.
$\hat{S}(z)$	Estimated secondary-path model exported by Module 5.
$x_f[n]$	Reference filtered through $\hat{S}(z)$ for filtered-x adaptation.
scenario_id	Stable identifier for one acoustic/signal condition.
run_id	Identifier for one reproducible algorithm execution.
window_id	Stable identifier for one exported fixed-length data window.
manifest	Machine-readable inventory linking signals, metadata, metrics, paths, and provenance.
clean_target[n]	Optional controlled/synthetic target used for supervised evaluation; never required for all real recordings.

Shared engineering rule: raw signals, path models, algorithm outputs, evaluation metrics, and AI-ready windows must remain distinguishable. A downstream model must always be able to determine where a sample came from, which scenario produced it, and which classical baseline generated the residual.

3. MODULE 6 — Recorded/Measured Acoustic ANC Integration

Objective: Extend the validated digital ANC pipeline so that the same controller and secondary-path interfaces can operate on reproducible recorded or measured data, while retaining simulation as a separate test mode.

Problem statement

Modules 4–5 already establish the controller-side architecture and secondary-path model artifact. The next requirement is to prove that the software is not tied only to internally generated signals. Module 6 introduces a data-source boundary: synthetic signals, prerecorded microphone/audio data, and future live streams must enter through explicit adapters rather than through separate algorithm implementations.

```
Data source
■■■ synthetic experiment
■■■ recorded reference / microphone data
■■■ future live audio source
↓
common signal contract + metadata
↓
Module 5 SecondaryPathModel ■(z)
↓
ANC replay engine
↓
FxLMS / FxNLMS
↓
residual e[n] + controller history + run artifact
```

Scope boundary: Module 6 is an offline/reproducible integration module. It does not require real-time streaming, low-latency device I/O, or embedded deployment. Those belong after the non-AI and AI foundations are validated.

3A. Recorded/Measured Data Interface

- Define a reusable loader/adaptor for WAV/array-based recordings and existing synthetic signals.
- Use one canonical floating-point signal convention internally and record original sample rate, channel count, duration, scaling/normalization policy, and provenance.
- Support configurable channel selection and explicit mono conversion where required.
- Reject empty, non-finite, incompatible, or sampling-rate-inconsistent inputs.
- Do not silently resample. Resampling, if supported, must be explicit and recorded in metadata.
- Preserve simulation and recorded modes as separate scenario types so algorithm failures are attributable to the correct layer.

Recommended reusable interfaces

```
AcousticRecording
samples
sample_rate
channel_metadata
source_type: synthetic | recorded
provenance

ANCScenario
reference_x
disturbance / measured_error_input
secondary_path_model: SecondaryPathModel
optional true path (simulation validation only)
metadata + scenario_id
```

Output: a stable scenario object that can be consumed by the ANC replay engine without knowing whether the source was synthetic or recorded.

3B. Calibration and Alignment

- Provide explicit checks for reference/error length compatibility and timing alignment.
- Support known or estimated fixed delay metadata; do not hide delay correction inside the controller.
- Record gain/scaling conventions so controller behavior can be reproduced.
- Where controlled recordings provide probe-response data, load the Module 5 identified model rather than reusing the hidden true path.
- Keep true-path information, if present in synthetic validation, outside the estimator/controller input path.

3C. Offline ANC Replay Engine

Implement one experiment-level replay path that reuses the existing Module 4 ANC plant/controller infrastructure. The engine should consume an ANCScenario, a controller configuration, and a secondary-path model, then produce the same style of output artifact for every data source.

Required cases

- Controller-disabled / no-control baseline.
- FxLMS using a validated or identified $\hat{S}(z)$.
- FxNLMS using the same scenario.
- Synthetic regression case where a known true $S(z)$ is available only for controlled validation.
- At least one recorded/measured scenario if recordings are available.

Important rule

For recorded data, the project must not pretend that an unavailable physical quantity is known. If a real acoustic recording lacks a clean primary disturbance or a true secondary path, the artifact must mark that quantity as unavailable rather than fabricating it.

3D. Module 6 Required Outputs

Artifact	Purpose
scenario_manifest.json	Scenario identity, source, timing, paths, provenance, and configuration.
reference_x.npy	Reference signal used by the controller.
error_input_or_disturbance.npy	Controlled disturbance/error-domain input with availability metadata.
controller_output_y.npy	Generated controller output.
residual_error_e.npy	Residual/error sequence.
filtered_reference.npy	Filtered-x reference where applicable.
controller_coefficients.npy	Final coefficient vector.
coefficient_history.npy	Controller adaptation trajectory.
secondary_path_model_used.npz	Portable identified path model or reference to immutable model artifact.
anc_replay_summary.json	Run configuration, powers, reduction metrics, and status.

3E. Module 6 Experiments, Visualizations, and Acceptance

Required experiments

- Synthetic regression replay using the existing controlled ANC setup.
- Identified-secondary-path replay using the Module 5 model artifact.
- At least one non-synthetic recorded/measured scenario when such data are available.
- Delay/alignment sensitivity case with controlled metadata changes.
- No-control vs FxLMS vs FxNLMS comparison under the same scenario.

Required visualizations

- Reference and residual time-domain views.
- No-control/input error versus controlled residual.
- Residual/error power over adaptation time.
- Controller coefficient trajectories.
- Frequency-domain attenuation or residual spectrum where applicable.
- Alignment/delay diagnostic for recorded scenarios.

Module 6 deliverable

A reproducible ANC replay/calibration layer that accepts common scenario objects, uses the Module 5 secondary-path model artifact, reuses the Module 4 filtered-x controller, and produces comparable ANC outputs for synthetic and recorded/measured inputs.

Acceptance conditions

- All data adapters and scenario validators pass unit tests.
- Existing synthetic ANC regression behavior remains reproducible.
- Module 5 path models can be loaded without experiment-specific code.
- The controller implementation is not duplicated for recorded data.
- Recorded and synthetic provenance are preserved in every summary/manifest.
- At least one end-to-end scenario runs entirely from configuration and saves the complete artifact set.

4. MODULE 7 — Standardized Evaluation & AI-Ready ANC Dataset

Objective: Convert the completed classical ANC pipeline into a standardized benchmark and dataset interface so that Module 8 can train and evaluate AI models using clean, versioned, reproducible inputs and explicit DSP baselines.

Problem statement

By this stage the project can generate signals, estimate Wiener benchmarks, adapt LMS/NLMS filters, run a feedforward ANC plant, identify a secondary path, and replay ANC scenarios. AI should not be connected directly to arbitrary experiment folders. Module 7 therefore standardizes scenarios, metrics, baseline outputs, windowing, splits, and metadata.

```
Module 6 run artifacts
+
Modules 1-5 baseline artifacts
↓
scenario standardization
↓
common evaluation metrics
↓
fixed-length windows + metadata
↓
train / validation / test manifests
↓
AI-ready dataset interface for Module 8
```

Scope boundary: Module 7 does not train a neural network or implement learned suppression. It prepares the data and evaluation contract against which Module 8 must compete.

4A. Standardized Scenario Matrix

Define scenarios as data/configuration, not as separate code paths. At minimum, support the following scenario axes:

Axis	Examples
Signal class	Broadband, colored/correlated, tonal/narrowband, transient/burst.
Input level	Controlled low/nominal/high amplitude or SNR conditions.
Path condition	Nominal, delay variation, gain variation, identified-model mismatch.
Source type	Synthetic, recorded/measured.
Controller	No control, LMS/NLMS baseline where meaningful, FxLMS, FxNLMS.
Secondary-path model	Perfect simulation model, identified model, controlled mismatch.

Each scenario must have a stable **scenario_id**, a configuration snapshot, a random seed where applicable, and provenance linking it to source recordings and secondary-path artifacts.

4B. Evaluation Metrics

- Initial and final residual/error power.
- Noise-reduction / attenuation in dB relative to the appropriate no-control baseline.
- Time-varying convergence or settling behavior.
- Segment-level and full-run MSE/RMSE where a controlled target exists.
- Frequency-band attenuation for scenarios where spectral comparison is meaningful.
- Controller stability indicators such as divergence/non-finite detection and coefficient growth.
- Secondary-path mismatch sensitivity when applicable.

Optional speech/intelligibility metrics may be included only when the data and target definitions genuinely support them; they must not be reported for synthetic scenarios where their assumptions are invalid.

4C. AI-Ready Dataset Contract

The dataset export must preserve the validated classical pipeline while exposing model-ready windows. The minimum unit of export is a fixed-length window plus metadata.

Required fields per window

Field	Requirement
reference_x	Required when available; aligned controller reference window.
baseline_residual	Residual/error before learned enhancement, with algorithm identity.
controller_output_y	Optional but recommended control signal context.
filtered_reference	Optional DSP context for filtered-x-aware models.
target	Optional controlled/synthetic target; unavailable values must be explicit.
sample_rate	Required.
scenario_id / run_id / window_id	Required provenance keys.
algorithm_metadata	Controller type and key configuration reference.
path_metadata	Secondary-path model identity and relevant mismatch flags.
source_type	Synthetic or recorded/measured.

Windowing rules

- Make window length and hop configuration-driven.
- Do not randomly split overlapping windows from the same recording across train and test sets.
- Perform splits at scenario/recording/run level to reduce leakage.
- Store dataset-level statistics using training data only.
- Any normalization used for AI must be saved as a reproducible artifact.

4D. Baseline Package and Required Outputs

For every scenario selected for the AI-ready dataset, preserve the non-AI baseline. The AI phase must be able to answer whether learning improves on the best validated classical output rather than only comparing against raw noise.

Artifact	Purpose
benchmark_manifest.json	Inventory of scenarios, runs, versions, and availability.
metrics_table.csv / parquet	One standardized row per run/scenario.
scenario_config snapshots	Exact reproducibility.
baseline_outputs/	Pointers or immutable copies of classical ANC outputs.
dataset_manifest.json	Train/validation/test membership and window inventory.
windows/	Model-ready arrays/tensors in a stable format.
normalization.json	Training-derived scaling statistics and policy.
evaluation_summary.json	Aggregate baseline metrics and acceptance results.
plots/	Standardized comparison figures.

4E. Required Visualizations

- Scenario-level no-control versus controlled residual comparison.
- Algorithm comparison across common scenarios.
- Noise reduction distribution across scenarios.
- Convergence/stability summaries.
- Frequency-domain attenuation summaries where applicable.
- Dataset split summary by scenario/source type, not just by number of windows.

4F. Module 7 Deliverable and Acceptance Conditions

Module 7 deliverable

A reproducible evaluation and AI-ready dataset package that turns Modules 1–6 into a standardized collection of scenario definitions, classical baseline outputs, metrics, model-ready windows, split manifests, and provenance. Module 8 should be able to load this package without importing experiment-specific scripts.

Acceptance conditions

- Every exported run has a stable scenario_id/run_id and reproducible configuration.
- No-control and classical ANC baselines are preserved for AI comparison.
- Metrics are computed through reusable evaluation functions, not hand-written per experiment.
- Train/validation/test splits prevent obvious recording/scenario leakage.
- Missing clean targets or unavailable physical quantities are represented explicitly.
- Dataset loading works without depending on Module 6 experiment internals.
- At least one end-to-end dataset build runs from configuration and produces a complete manifest.
- Unit tests plus at least one integration/acceptance test validate export, reload, metrics, and split integrity.

5. Module Connection

```
Module 5
SecondaryPathModel ■(z)
↓
Module 6
Recorded/Measured ANC Scenario → FxLMS/FxNLMS Replay → Residual Artifacts
↓
Module 7
Scenario Matrix → Metrics → Baselines → Windows → Manifests
↓
Module 8
AI model training and inference on organized ANC inputs
```

6. End-of-Pre-AI Phase Milestone

At the end of Modules 6–7, the project should have completed the classical, non-AI software foundation and should be ready to introduce learning without losing interpretability or baselines.

Required phase-level evidence

- A reproducible ANC replay using the identified secondary-path artifact from Module 5.
- Clear separation between synthetic and recorded/measured data paths.
- A controller-disabled baseline and filtered-x baselines under standardized scenarios.
- Reusable evaluation metrics and algorithm comparison.
- Saved numerical artifacts, metadata/manifests, and visual diagnostics.
- An AI-ready window/dataset interface with provenance and leakage-aware splits.
- Unit tests and at least one end-to-end acceptance test for each major module.

7. Forward Interface — Module 8 Begins AI

Module 8 is the first learned stage. It should consume the stable Module 7 dataset contract and be evaluated against the preserved classical ANC baselines. The AI component should initially operate as residual-noise suppression or learned enhancement around the existing DSP/ANC pipeline rather than replacing the validated controller immediately.

```
AI INPUTS (from Module 7)
reference / baseline residual / optional controller context / metadata
↓
Module 8 learned model
↓
enhanced residual / suppression estimate
↓
same evaluation interface + comparison to classical baselines
```

Engineering rule: AI is added only after the inputs, targets, baselines, metrics, and provenance are stable. Every learned result must remain independently testable and comparable against explicit non-AI conditions.